

Medical AI Reviewer

Design Document

A Multi-Agent RAG Architecture for Post-Payment Claim Review Support



Prepared by:

Shirisha Dhannini & Kapil Suthraye

Programme: EY Generative AI Training

Organisation: Ernst & Young (EY) | Hyderabad, India

Table of Contents

1. Project Overview & Problem Statement
2. Market Research & Opportunity
3. Proposed Solution
4. Project Type Classification
5. High-Level Architecture
6. Tech Stack & Technology Choices
7. Agent Design
8. Data Sources & Schema
9. Guardrails & Safety
10. Observability & Monitoring
11. Evaluation Metrics
12. Success Metrics

1. Project Overview & Problem Statement

Insurance companies conduct post-payment claim reviews to verify that providers billed correctly and that medical records support the payments already made. This process is performed manually by clinical nurse reviewers, who must navigate lengthy patient records — often exceeding 1,000 pages — to locate relevant clinical evidence and assess whether billed services are adequately documented and justified.

Key Challenges:

- Medical records frequently exceed 1,000 pages, making manual review slow and cognitively demanding.
- A nurse reviewer can process only 4–5 claims per day, while insurers may receive thousands of claims for review daily.
- The process relies entirely on trained clinical professionals, making it expensive and difficult to scale.
- Delayed reviews slow down recovery actions and increase the risk of uncollected overpayments.
- Manual review of large, unstructured records leads to inconsistent outcomes across different reviewers.

2. Market Research & Opportunity

The healthcare claims management and review market is experiencing significant growth, driven by rising claim volumes, increasing complexity of medical coding (ICD-10/CPT), and growing pressure on payers to reduce overpayment leakage. AI-assisted review tools represent one of the highest-ROI applications of Generative AI in healthcare payer operations.

Metric	Insight
Daily claim review demand	Thousands of claims per insurer per day vs. 4–5 claims per nurse
Average medical record length	Often exceeds 1,000 pages per patient episode
Cost driver	High dependence on trained clinical reviewers; expensive to scale
Recovery opportunity	AI first-pass reduces missed discrepancy risk and speeds recovery actions
Competitive landscape	Emerging space — RAG + clinical NLP solutions beginning to appear in payer tech
Regulatory context	Human-in-the-loop mandatory; AI cannot auto-deny or auto-recoup payments

Opportunity Summary:

- Reduce per-claim review cost by augmenting nurses with AI-prepared evidence packages.
- Scale review capacity without proportional headcount increases.
- Improve consistency and auditability of review decisions.
- Enable focus on high-complexity cases by automating initial evidence extraction.

3. Proposed Solution

Medical AI Reviewer is a Retrieval-Augmented Generation (RAG) system designed to support post-payment claim review. It reads patient medical records, finds evidence relevant to each billed claim item, identifies possible discrepancies, and presents a concise, evidence-backed summary to the nurse reviewer.

The system shifts the nurse's role from *reading everything manually* to *reviewing AI-prepared evidence and making the final clinical and compliance decision*.

Core Capabilities:

- Ingest and process large medical records (PDF, DOC, scanned images via OCR).
- Chunk, embed, and index documents in a vector store for semantic retrieval.
- Retrieve the most relevant passages for each claim line item using hybrid search.
- Extract supporting or contradicting clinical evidence for billed services, codes, and dates.
- Detects discrepancies: missing documentation, unsupported services, coding inconsistencies, date mismatches.
- Generate concise, reviewer-friendly summaries with page-level evidence citations.
- Provide an interactive Q&A interface for follow-up questions on the claim or record.
- Capture reviewer feedback to improve system performance over time.

One-line summary: Medical AI Reviewer helps insurance nurses review claims faster by reading large medical records, finding supporting evidence, and flagging possible discrepancies.

4. Project Type Classification

Primary Type: Medical Document Summarizer + Clinical Evidence Recommender

- **Summarization:** Converts lengthy, unstructured medical records into concise, structured reviewer-friendly outputs.
- **Evidence Recommendation:** Surfaces the most relevant clinical passages and flags discrepancies to help nurses validate claims faster.

This classification places the system in the category of **AI-augmented decision support** — not autonomous decision-making. The AI prepares, organises and highlights; the nurse decides.

5. High-Level Architecture

The system is built on a multi-agent RAG pipeline coordinated by an Orchestrator/Supervisor agent. External data (medical records, claims, policy rules) flows through an Ingestion & RAG Retriever agent, which fetches semantically relevant evidence from a hybrid vector database. This evidence is passed to the Evidence Extraction & Discrepancy Detection agent, which performs extraction, reasoning, and policy checks. The Summary Generation agent produces nurse-ready outputs, which are reviewed by the human-in-the-loop layer. LangSmith provides end-to-end observability across all LLM workflows.

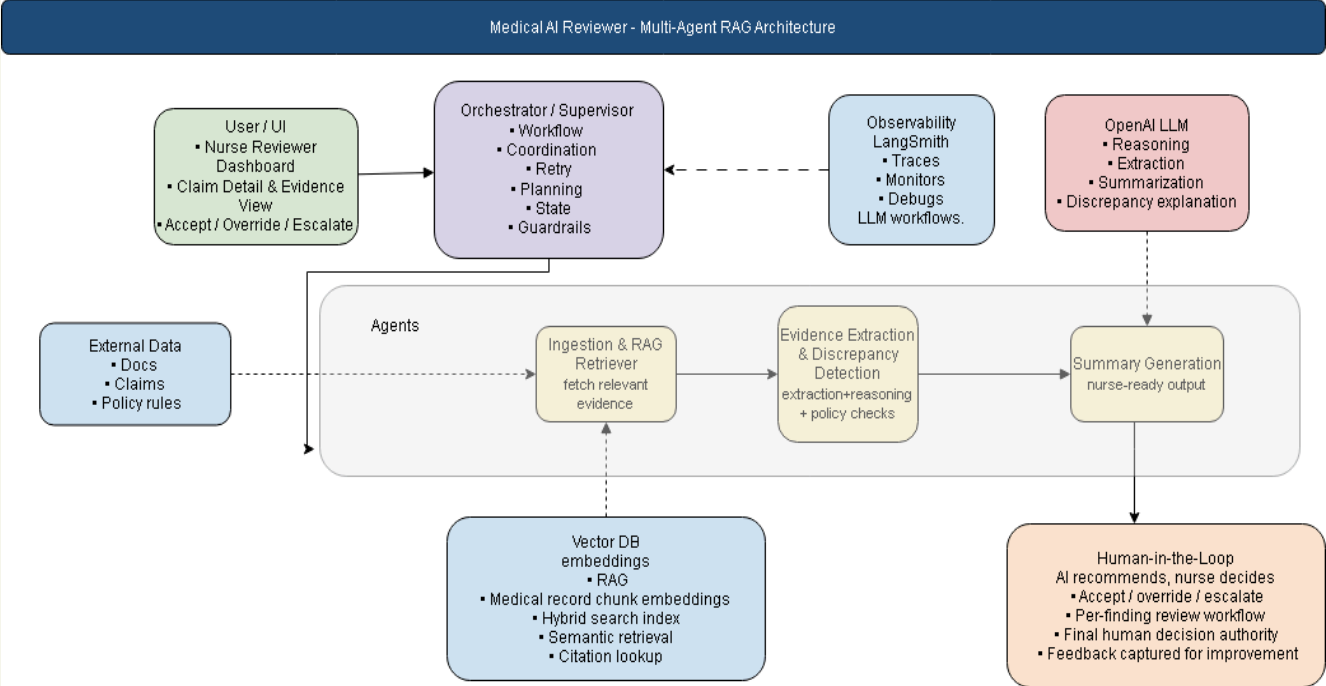


Figure 1: Medical AI Reviewer — Multi-Agent RAG Architecture

Component Summary:

Component	Role
User / UI (Streamlit)	Nurse Reviewer Dashboard — Claim Detail, Evidence View, Accept/Override/Escalate
Orchestrator / Supervisor	Workflow coordination, retry logic, planning, state management, guardrails
Ingestion & RAG Retriever Agent	Document ingestion, chunking, embedding, hybrid semantic + keyword retrieval
Evidence Extraction & Discrepancy Detection	Clinical extraction, reasoning, policy rule checks, discrepancy flagging
Summary Generation Agent	Produces structured nurse-ready output with citations and confidence indicators
Vector DB (FAISS)	Stores chunk embeddings; supports hybrid search index, semantic retrieval, citation lookup
Human-in-the-Loop	Nurse accepts, overrides, or escalates; feedback captured for continuous improvement
Observability (LangSmith)	Traces, monitors, and debugs all LLM workflow steps
OpenAI LLM (GPT-4o)	Reasoning, extraction, summarisation, discrepancy explanation

Component	Role
External Data Sources	Medical record docs, claims data, payer policy rules

6. Tech Stack & Technology Choices

Each technology in the stack was selected for a specific reason. The table below documents the choice, its purpose, and its trade-offs.

Layer	Technology	Purpose	Pros	Cons / Trade-offs
Frontend	Streamlit	Reviewer UI / prototype dashboard	Rapid prototyping; Python-native; no frontend expertise needed	Limited customisation; not production-grade UI; scalability constraints
Backend	Python	Core application and AI pipeline logic	Rich AI/ML ecosystem; LangChain-native; rapid development	GIL limits true parallelism; async patterns add complexity
Doc Parsing	PyPDF / Unstructured	Extract and process text from medical records	Handles diverse formats (PDF, DOCX, scanned); Unstructured covers complex layouts	OCR quality depends on scan resolution; Unstructured can be slow on very large files
Embeddings	HuggingFace BGE	Convert document chunks into vector embeddings	High-quality semantic embeddings; open-source; domain-adaptable	Requires local inference resources; updates need re-embedding pipeline
Vector Store	FAISS	Store and retrieve semantic search results	Fast approximate nearest-neighbour search; in-memory; battle-tested	Not distributed; no built-in persistence; limited metadata filtering vs managed solutions
AI / LLM	GPT-4o / Claude / Llama	Evidence analysis, summarisation, discrepancy detection	GPT-4o: leading reasoning; Claude: long-context; Llama: on-premise option	GPT-4o: cost and latency; Claude: API dependency; Llama: infrastructure overhead
RAG Framework	LangChain	Orchestrate retrieval and LLM workflow	Mature ecosystem; agent tooling; strong community and docs	Abstraction overhead; rapid API changes; debugging multi-agent chains can be complex
Storage	Local Files / PostgreSQL	Store uploaded files, metadata, review data	PostgreSQL: ACID compliance, relational structure; Local: simple for prototype	Local files: not scalable; PostgreSQL: requires schema management and ops overhead
Observability	LangSmith	Trace prompts, retrieval, and model outputs	Native integration; debugging; workflows; LangChain prompt evaluation	Adds latency overhead; data leaves environment (cloud-hosted); cost at scale
Evaluation	RAGAS	Evaluate retrieval and answer quality	Purpose-built for RAG evaluation; covers faithfulness, relevancy, context recall	Requires ground-truth data for some metrics; adds pipeline complexity

7. Agent Design

Orchestrator / Supervisor Agent

Central controller of the multi-agent pipeline.

- Receives claim review task from the UI layer.
- Plans the sequence of sub-agent calls based on task type.
- Manages state across the pipeline run.
- Handles retry logic and failure recovery.
- Enforces guardrails before passing outputs downstream.
- Returns final output to the reviewer dashboard.

Ingestion & RAG Retriever Agent

Handles all document processing and evidence retrieval.

- Ingests medical records via PyPDF / Unstructured.
- Chunks documents into semantically meaningful segments.
- Generates embeddings via HuggingFace BGE.
- Stores chunks and embeddings in FAISS vector store.
- Performs hybrid search (semantic + keyword) for each claim query.
- Returns ranked, relevant passages with page-level metadata.

Evidence Extraction & Discrepancy Detection Agent

Performs clinical reasoning over retrieved evidence.

- Extracts clinical facts relevant to each billed service.
- Cross-references extracted evidence against claim line items.
- Applies payer policy rules to identify unsupported services.
- Detects date mismatches, unit inconsistencies, and coding anomalies.
- Produces structured discrepancy flags with evidence citations.

Summary Generation Agent

Produces the nurse-ready review output.

- Synthesises extracted evidence and discrepancy flags.
- Generates a concise, structured summary for each claim.
- Includes page-level citations for every finding.
- Formats output for the reviewer dashboard.
- Highlights confidence level for each finding.

8. Data Sources & Schema

Data Source	Format	Content	Usage
Medical Records	PDF, DOC, scanned image	Patient history, clinical notes, lab results, discharge summaries	Primary RAG input
Claim Line Items	Structured / CSV / EHR export	Billed CPT/ICD codes, dates of service, units, provider info	Query formulation for retrieval
Payer Policy Rules	Document / rule engine	Coverage criteria, medical necessity guidelines, coding policies	Policy check in discrepancy agent
Reviewer Feedback	PostgreSQL	Accept / override / escalation decisions, corrections	Feedback loop for prompt and rule improvement
Audit Logs	PostgreSQL	System actions, retrieved evidence, AI outputs, reviewer decisions	Compliance and traceability

9. Guardrails & Safety

- **No automated decisions:** The system will not auto-deny, auto-adjust, or auto-recoup payments under any circumstance.
- **Human-in-the-loop mandatory:** Every AI output requires nurse review and explicit accept / override / escalate action.
- **Confidence thresholds:** Findings below a defined confidence threshold are flagged for mandatory human review rather than presented as conclusions.
- **Hallucination prevention:** All summaries include page-level citations; the system is instructed to say 'evidence not found' rather than infer.
- **PHI / data security:** Medical records contain Protected Health Information (PHI); access controls, audit logging, and data handling must comply with applicable regulations.
- **No clinical advice:** The system is scoped strictly to claim review support — it does not provide diagnosis, treatment recommendations, or clinical guidance.
- **Scope enforcement:** The orchestrator enforces that agents operate only within their defined scope; out-of-scope queries are rejected with an explanation.
- **Reviewer override always wins:** Any reviewer correction or override is treated as ground truth and logged for system improvement.

10. Observability & Monitoring

LangSmith is used as the primary observability layer, providing end-to-end tracing of all LLM workflow steps, prompt versions, retrieval results, and agent outputs.

Observability Concern	Tool / Approach
LLM prompt tracing	LangSmith — full prompt/response logging per agent step
Retrieval quality monitoring	LangSmith + RAGAS — track context relevance and recall
Answer faithfulness	RAGAS — measure whether outputs are grounded in retrieved evidence
Latency tracking	LangSmith — per-step and end-to-end latency monitoring
Error and retry tracking	Orchestrator state machine + LangSmith traces
Reviewer feedback capture	PostgreSQL feedback table — corrections, overrides, escalations

Observability Concern	Tool / Approach
Audit trail	PostgreSQL audit_logs table — all system actions and decisions

11. Evaluation Metrics

Evaluation is performed using RAGAS, a framework purpose-built for assessing RAG pipeline quality. The following metrics are tracked continuously:

Metric	Definition	Target
Context Recall	Fraction of relevant evidence passages successfully retrieved from the record	≥ 0.85
Context Precision	Fraction of retrieved passages that are actually relevant to the claim query	≥ 0.80
Answer Faithfulness	Degree to which the generated summary is grounded in retrieved evidence (no hallucination)	≥ 0.90
Answer Relevancy	How well the generated summary answers the claim review question	≥ 0.85
Evidence Citation Accuracy	Percentage of citations that correctly reference the source page and passage	≥ 0.90
Discrepancy Detection Rate	Percentage of known discrepancies correctly flagged by the system	≥ 0.80
False Positive Rate	Percentage of flagged discrepancies that are incorrect (nurse-verified)	≤ 0.15
End-to-End Latency	Time from claim submission to nurse-ready summary generation	≤ 30 seconds
Reviewer Override Rate	Percentage of AI findings overridden by nurse reviewers (quality proxy)	Monitor; target downward trend
Feedback Capture Rate	Percentage of review sessions where reviewer feedback is recorded	≥ 0.90

12. Success Metrics

These are the business and operational outcomes that define whether the system has achieved its goals:

Success Metric	Target	Measurement Method
Reduction in average claim review time	~70% reduction vs. baseline	Compare pre/post nurse review time logs
Evidence retrieval accuracy	High accuracy for claim-related queries	RAGAS context recall and precision
Summary generation response time	Within a few seconds per claim	LangSmith end-to-end latency traces
Reviewer satisfaction	Improvement in usefulness and trust scores	Post-session reviewer feedback surveys
Claim review throughput	Significant increase in claims reviewed per nurse per day	Operational productivity tracking
System stability under load	Stable processing of multiple records in sequence	TC-08: sequential multi-record test

Success Metric	Target	Measurement Method
Audit compliance	100% of AI outputs logged with evidence citations	Audit log completeness checks